

10-Gen Campaign Analysis + Spec Update

Eight of ten generations were viable. The record compares code quality across survivors and proposes two specification changes; its price estimates remain historical source claims.

Real project record · retrospective HTML adaptation

Original text from Cambrian master at a4d2181c488b. The source is preserved in full below and in the linked Markdown. New headings, metadata summaries and visual summaries are an adaptation, not newly observed evidence. Pinned source.

SOURCE AND ADAPTATION

Record provenance

SOURCE RECORD	journal-2026-04-04b	PROJECT	Cambrian
SOURCE DATE	2026-04-04	SOURCE SIGNATURE	2026-04-04T10:30:00+04:00
OPENED TIME	Not recorded	SOURCE AUTHORS	lispmeister + AI assistant
TOOLS (SOURCE)	Claude Opus 4.6 (plan mode) / Claude Sonnet 4.6, Python 3.14.3, Docker Desktop	STATE (SOURCE)	fead6b8 (source Commit / Witness field; no separate witness identified)
INDEPENDENT WITNESS	Not separately recorded	ADAPTATION	Codex · 2026-09-06 · retrospective
HANDLING	Public repository source; underlying artifacts not imported		

ORIGINAL SOURCE SECTION

Session Goals

- Differential code quality analysis across 8 viable gens (39, 42–48)
- Cost comparison across models
- Apply two spec improvements from analysis findings

SOURCE SECTION · MIXED EVIDENCE AND INTERPRETATION

Campaign Results: Gens 39–48

Viability: 8/10 (80%) — best rate observed to date (prior campaigns dominated by infra failures).

GEN	OUTCOME	FAILURE STAGE	SUMMARY
39	promoted	—	viable

GEN	OUTCOME	FAILURE STAGE	SUMMARY
40	failed	test	4/71 tests failed: <code>get_model()</code> ignores env vars; <code>parse_files()</code> too permissive
41	failed	start	73/73 tests pass; process exits with code 0 before binding port
42	promoted	—	viable
43	promoted	—	viable
44	promoted	—	viable
45	promoted	—	viable
46	promoted	—	viable
47	promoted	—	viable
48	promoted	—	viable

SOURCE SECTION • MIXED EVIDENCE AND INTERPRETATION

Differential Code Quality Analysis

All 8 viable gens are type-annotated, use f-strings, pathlib, Pydantic, asyncio.sleep, and env-var config. These are stable outputs — the LLM reliably produces idiomatic Python on these dimensions.

Comparison Table

DIMENSION	39	42	43	44	45	46	47	48
Type annotations	✓	✓	✓	✓	✓	✓	✓	✓
f-strings	✓	✓	✓	✓	✓	✓	✓	✓
Pathlib	✓	✓	✓	✓	✓	✓	✓	✓
Pydantic models	✓	✓	✓	✓	✓	✓	✓	✓
asyncio.sleep	✓	✓	✓	✓	✓	✓	✓	✓
time.monotonic()	✗	✗	✗	✗	✓	✓	✗	✗
Specific exceptions	✗	✓	partial	✗	✗	partial	✗	✗
Structlog correct	✓	✓	✗	✓	✓	✓	✗	✓
Backoff strategy	✗	✓	✗	✗	✗	✗	✗	✗

Key Findings

time.time() vs time.monotonic(): 6/8 gens use `time.time()` for uptime. Gens 45 and 46 correctly use `time.monotonic()`. The system clock can jump backward on NTP adjustments — `time.time()` is wrong here.

Exception handling: Only Gen-42 catches `aiohttp.ClientError` specifically. Most catch bare `except Exception`. Gens 43 and 46 at least catch `FileNotFoundError` for the spec file.

Backoff strategy: Only Gen-42 implements exponential backoff in the Supervisor client:

```
backoff = 1.0
except aiohttp.ClientError:
    await asyncio.sleep(backoff)
    backoff = min(backoff * 2, 60.0)
```

Structlog anti-patterns (gens 43, 47):

```
# Gen-43 (wrong – event duplicated as kwarg):
log.error("anthropic_key_missing", event="anthropic_key_missing")
# Gen-47 (wrong – non-standard kwarg):
log.error("missing_api_key", event_msg="ANTHROPIC_API_KEY not set...")
```

Overall Ranking

RANK	GEN	REASON
1	42	Specific exceptions, exponential backoff, cleanest supervisor client
2	45/46	<code>time.monotonic()</code> , correct logging
3	39/44/48	Correct, no standout features
4	43/47	Structlog anti-patterns

SOURCE SECTION • MIXED EVIDENCE AND INTERPRETATION

Spec Updates Applied (v0.14.3 → v0.14.4)

Two SHOULD rules added based on analysis findings:

- `time.monotonic()` for uptime — added to `/stats` description. 6/8 gens got this wrong.
- Exponential backoff with code example — added to Implementation Requirements under the `aiohttp.ClientSession` bullet. Only 1/8 gens implemented it despite being documented in the failure table. A concrete code example should improve the rate.

SOURCE SECTION • MIXED EVIDENCE AND INTERPRETATION

Cost Comparison Reference

Historical source estimates at 2026-04-04 rates; not current verified pricing.

Pricing at 2026-04-04 rates. Token counts from two sources: gen-32/35 (single-shot, no feedback context) and the gens 39–48 campaign (with `failed_context` feedback loop adding ~1k input tokens per gen).

MODEL	INPUT \$/M	OUTPUT \$/M	TYPICAL IN	TYPICAL OUT	COST/GEN	COST/10 GENS
claude-sonnet-4-6 (fresh)	\$3.00	\$15.00	52k	18k	~\$0.43	~\$4.30
claude-sonnet-4-6 (w/ feedback)	\$3.00	\$15.00	61k–71k	17k–19k	~\$0.45–\$0.50	~\$4.70
claude-opus-4-6 (fresh)	\$15.00	\$75.00	77k	36k	~\$3.83	~\$38.30
gpt-5.3-codex (estimated)	\$1.75	\$14.00	~52k–71k	~18k	~\$0.34–\$0.37	~\$3.60

Notes:

- "Fresh" = no `failed_context` (first gen of a campaign or standalone run)
- "w/ feedback" = `failed_context` from prior gen adds ~1k input tokens per gen; cumulative over a campaign
- Opus uses ~1.5–2× more tokens than Sonnet per gen (larger output, longer reasoning)
- Sonnet is the default model; use Opus only when explicitly requested
- GPT-5.3-Codex pricing sourced from Opencode Zen; no empirical run yet — verify before each run

What the source makes visible

KIND Retrospective source summary	CREATOR Codex · 2026-09-06	SUPPORT Retained original Markdown	NEW MEASUREMENTS None
Reported outcome groups (not chronological steps) 10 generations: 39–48 8 promoted 1 test-stage failure 1 start-stage failure Outcomes and quality judgments are reported by the source; underlying generated programs and run exports are not retained in this sample. Cost estimates are dated 2026-04-04 and are not verified current pricing. A successful test suite did not ensure successful startup.			

Source evidence and retention

SOURCE	RETENTION	INTEGRITY
Complete original Markdown	Unchanged local bytes	c389f43890f8b80fef568a4e79dff180da700983d6eac8de922b7388cc087d8e
Pinned upstream record	External; requires network when followed	a4d2181c488bc5f0ff329438541cba51e8543fc8

What this rendering cannot establish

Outcomes and quality judgments are reported by the source; underlying generated programs and run exports are not retained in this sample. Cost estimates are dated 2026-04-04 and are not verified current pricing. A successful test suite did not ensure successful startup.

Source recommendations remain source recommendations. No new historical observations, predictions, witness signatures or experiment results were created. The original signature is reproduced below as attribution.

ORIGINAL SOURCE CLOSURE

Signature and next entry, as recorded

Signed: lispmeister + AI assistant **Date:** 2026-04-04T10:30:00+04:00

Participants & Tools: Claude Opus 4.6 (plan mode) / Claude Sonnet 4.6, Python 3.14.3, Docker Desktop **Commit /**

Witness: fead6b8 **Related Specs / Beads:** CAMBRIAN-SPEC-005 v0.14.4, BOOTSTRAP-SPEC-002 v0.9.1

Next journal entry: journal-2026-04-04c.md (use TEMPLATE.md)

HTML adaptation: Codex · 2026-09-06; historical work not rerun

Unchanged Markdown source